



This member-only story is on us. [Upgrade](#) to access all of Medium.

★ Member-only story

Claude skills for Business Analysts and Product Managers

The Claude Code way: reusable skills, slash commands, and tool-connected workflows. Most BA and PM teams have already tried GenAI for “write me a PRD” or “summarize this meeting.” The initial results are impressive, but they often fail to scale because the work is not repeatable.

9 min read · Feb 2, 2026



Ravindra Satyanarayana

Follow



Listen



Share

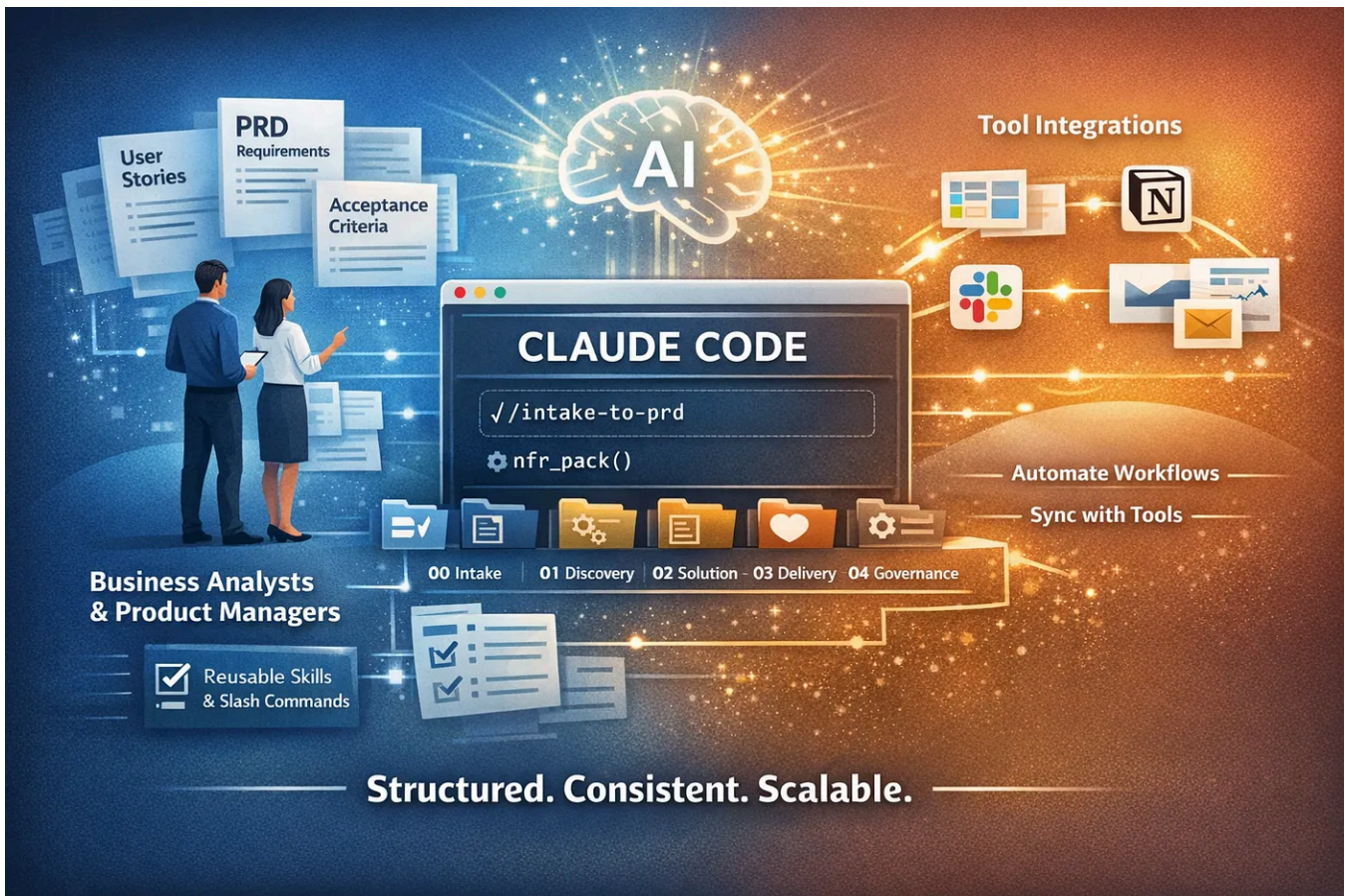


More

Claude Code changes the game because it treats your product work like an executable system:

- it is **file-aware** (works inside a folder/repo of your artifacts),
- it supports **Skills** (structured, reusable routines you can invoke),
- it supports **slash commands** (repeatable mini-workflows),
- and it can connect to real tools via **MCP (Model Context Protocol)** for Jira/Confluence/Notion/Slack-style workflows.

What follows is a pragmatic, BA/PM-specific operating model.



Claude Code Skills for BA / Product Manager / Requirements Generation

1) Think “product workbench”, not “chat prompts”

In Claude Code, you get leverage when you create a **single folder (or repo)** that becomes your product workbench.

A recommended structure:

```
product-workbench/  
  00_intake/  
    meeting-notes/  
    stakeholder-emails/  
    raw-requirements/  
  01_discovery/  
    personas/  
    journey-maps/  
    problem-statements/  
    research-notes/  
  02_solution/  
    prds/  
    user-stories/  
    acceptance-criteria/  
    nfrs/  
  03_delivery/  
    release-notes/
```

```
cutover/  
support-readiness/  
04_governance/  
decision-records/  
risk-log/  
assumptions/  
.claude/  
commands/  
SKILL.md
```

Why this matters: Claude Code can **read and write across these artifacts**, so your PRD, stories, AC, NFRs, risk log, and decision records stay consistent over time — rather than being one-off text outputs.

2) Create BA/PM “skills” as first-class assets

Claude Code Skills are a formal, reusable way to encode “how your org does product and analysis work.” The Claude Code docs describe creating a `SKILL.md` to add instructions and invokable routines.

A strong BA/PM `SKILL.md` typically defines:

- your PRD template (sections, expected rigor, decision checkpoints)
- your story writing standard (INVEST, vertical slicing, explicit edge cases)
- your acceptance criteria standard (Given/When/Then + negative cases)
- your prioritization rubric (RICE/WSJF + constraints)
- your governance artifacts (decision record format, risk scoring)

Example `SKILL.md` (high-impact, minimal):

```
# ProductOps Skill Pack  
  
## Global rules  
- Always produce structured outputs in Markdown with headings.  
- Always include assumptions explicitly when info is missing.  
- Always create a "Questions for stakeholders" section.  
  
## Artifacts  
### PRD  
Use: 02_solution/prds/PRD_TEMPLATE.md  
Output: 02_solution/prds/<feature>_PRD.md
```

Stories

Output: 02_solution/user-stories/<feature>_stories.md

Standard: INVEST, vertical slices, include dependencies.

Acceptance Criteria

Output: 02_solution/acceptance-criteria/<feature>_ac.md

Standard: Given/When/Then + negative cases + data/permission cases.

NFRs

Output: 02_solution/nfrs/<feature>_nfr.md

Must include: performance, security, privacy, observability, accessibility.

This turns “prompting” into **repeatable operational capability**.

3) operationalize skills as slash commands

In practice, BAs and PMs benefit most from **slash commands** because they behave like *standard operating procedures*.

Claude Code supports storing commands in either a global folder or per-project
.claude/commands .

recommended BA/PM command set (practical and immediately useful)

1. /intake-to-prd

Converts meeting notes + raw requirements into a PRD draft, with open questions.

2. /prd-to-epics

Breaks PRD into epics/features, including dependencies and assumptions.

3. /epics-to-stories

Produces sprint-ready stories with vertical slices.

4. /stories-to-ac

Generates acceptance criteria + edge/negative cases.

5. /nfr-pack

Creates NFR checklist aligned to the feature and your platform.

6. /risk-log-update

Updates risk register based on new scope/unknowns.

7. /decision-record

Generates an ADR-style decision record for key trade-offs.

A real command example (`.claude/commands/intake-to-prd.md`):

You are acting as a senior Product Analyst.

Input sources to read:

- `00_intake/meeting-notes/`
- `00_intake/raw-requirements/`

Task:

- 1) Produce a PRD in `02_solution/prds/<feature>_PRD.md`
- 2) Include: problem statement, goals/non-goals, user personas, scope, flows, functional requirements, data requirements, NFRs, success metrics, rollout plan, risks, and open questions.
- 3) Explicitly separate: "facts from inputs" vs "assumptions".
- 4) End with "Questions for stakeholders" (prioritized).

This is where Claude Code becomes materially different from generic chat: you are building **institutional memory + repeatability**.

4) use “plan-first” execution to reduce chaos and rework

A consistent pattern in Claude Code best practices is to use a **plan-first** approach for complex tasks and then execute in controlled steps.

For BA/PM workflows, the equivalent is:

- **Plan:** confirm scope, unknowns, definitions, and artifact set
- **Execute:** generate PRD → review → derive stories → review → AC/NFRs → review
- **Validate:** consistency checks (terms, metrics, edge cases, dependencies)

If your team struggles with variability in Claude outputs, make “plan-first” a mandatory step in your command procedures.

5) connect Claude Code to your real tools with MCP (where it becomes enterprise-grade)

The step-change happens when Claude Code is not only writing Markdown files, but can also interact with your delivery ecosystem through **MCP tool connections**.

Claude Code's docs describe connecting tools via MCP servers.

What this enables for BA/PM teams:

- Create/update tickets in Jira-style systems
- Pull context from Confluence/Notion pages
- Summarize Slack threads into decisions and actions
- Create structured tasks in Asana-style tools

This “tool-connected” direction is also where Claude's broader ecosystem has been moving — interactive integrations via MCP are explicitly positioned as a way to work directly with business apps.

A pragmatic BA/PM adoption sequence:

1. Start with **offline artifacts** (PRDs, stories, AC, risk logs) in your workbench repo
2. Add **slash commands** to standardize production quality
3. Introduce **MCP** only when your quality bar and governance are stable
4. Then automate “last-mile” actions (ticket creation, updates, summaries)

This avoids the common failure mode: connecting tools too early and scaling inconsistent outputs.

6) real-life outputs: what BA/PM teams should expect weekly

If you implement the above, the “real life” outcome is not vague productivity. It is measurable operational uplift.

A BA can reliably produce:

- PRD v1 + questions in 60–90 minutes (including review cycles)
- consistent story slicing without missing edge cases
- traceability between requirements → stories → AC → NFRs

A PM can reliably produce:

- roadmap options with explicit assumptions

- decision records for trade-offs (cost, timeline, risk, UX impact)
- release readiness checklists aligned to scope and NFRs

And the team gains a shared “product ops muscle” because it is encoded in skills/commands rather than individual heroics.

7) governance: how to keep Claude useful without creating risk

For BA/PM use, governance is simple and enforceable:

- **No silent assumptions:** always label assumptions
- **Source traceability:** link claims back to input artifacts
- **Definition lock:** maintain a glossary to avoid term drift
- **Approval gates:** PRD approval before story generation, story approval before AC finalization

This is also aligned with Claude Code’s positioning as an agentic tool that can take action — meaning the governance discipline matters even more when you connect it to MCP tools.

Below is a ready-to-use, enterprise-grade BA / Product Manager Claude Code Command Pack, designed exactly for how you and similar senior practitioners actually work.

This is not conceptual. You can drop this into a repo today and start using it in Claude Code.

Claude Code BA / Product Manager Command Pack

(Production-ready, repeatable, governance-aligned)

How to use this

1. Create a repo or folder called `product-workbench`
2. Add the structure and files below
3. Open the repo in **Claude Code**
4. Invoke commands using `/command-name`

1. Recommended Folder Structure (Minimal but Powerful)

```
product-workbench/  
  00_intake/  
    meeting-notes/  
    stakeholder-inputs/  
    raw-requirements/  
  01_discovery/  
    personas/  
    journey-maps/  
    problem-statements/  
    research/  
  02_solution/  
    prds/  
    epics/  
    user-stories/  
    acceptance-criteria/  
    nfrs/  
  03_delivery/  
    release-notes/  
    rollout/  
    support-readiness/  
  04_governance/  
    decisions/  
    risks/  
    assumptions/  
    glossary/  
  .claude/  
    SKILL.md  
    commands/
```

2. SKILL.md — The Operating System for BA / PM Work

Create this file at:

.claude/SKILL.md

```
# Product & Analysis Skill Pack  
  
## Global Principles  
- Always separate facts from assumptions  
- Always highlight open questions explicitly  
- Prefer structured Markdown outputs  
- Never invent data, metrics, or constraints  
- Use enterprise-grade language (no casual tone)  
  
## Artifact Standards  
  
### PRD
```


Sections required:

- Problem statement
- Business goals & success metrics
- In-scope / out-of-scope
- Personas
- User flows
- Functional requirements
- Data requirements
- Non-functional requirements
- Risks & dependencies
- Rollout considerations
- Open questions

User Stories

- INVEST-compliant
- Vertical slices only
- Explicit dependencies
- Include edge cases

Acceptance Criteria

- Given / When / Then
- Negative and boundary cases mandatory
- Permissions, roles, and data states included

NFRs

Must include:

- Performance
- Security
- Privacy
- Reliability
- Observability
- Accessibility
- Compliance (where applicable)

Governance

- Decisions recorded as ADR-style notes
- Risks scored (Low / Medium / High)
- Assumptions tracked separately

3. Slash Commands (Drop-in Ready)

Create each file under:

`.claude/commands/`

3.1 `/intake-to-prd`

File: `.claude/commands/intake-to-prd.md`

You are acting as a Senior Business Analyst and Product Manager.

Inputs to read:

- 00_intake/meeting-notes/
- 00_intake/stakeholder-inputs/
- 00_intake/raw-requirements/

Task:

1. Produce a PRD in:
02_solution/prds/<feature>_PRD.md
2. Follow the PRD standard defined in SKILL.md
3. Clearly label:
 - Confirmed facts
 - Assumptions
4. Add a final section:
"Questions for Stakeholders" (prioritized)

Quality bar:

- No vague requirements
- No unstated assumptions
- Enterprise-grade clarity

3.2 /prd-to-epics

File: .claude/commands/prd-to-epics.md

You are acting as a Product Owner.

Input:

- A single PRD from 02_solution/prds/

Task:

1. Break the PRD into epics and features
2. Output to:
02_solution/epics/<feature>_epics.md
3. For each epic include:
 - Objective
 - Scope
 - Dependencies
 - Risks
 - Acceptance boundaries

Rules:

- No technical implementation details
- Focus on business capability slices

3.3 /epics-to-stories

File: .claude/commands/epics-to-stories.md

You are acting as a Delivery-focused Product Manager.

Input:

- 02_solution/epics/<feature>_epics.md

Task:

1. Generate sprint-ready user stories
2. Output to:
02_solution/user-stories/<feature>_stories.md
3. Each story must include:
 - User story format
 - Acceptance intent
 - Dependencies
 - Non-goals

Rules:

- Vertical slices only
- Avoid technical task breakdowns

3.4 /stories-to-ac

File: .claude/commands/stories-to-ac.md

You are acting as a Quality-aware Business Analyst.

Input:

- 02_solution/user-stories/<feature>_stories.md

Task:

1. Generate acceptance criteria for each story
2. Output to:
02_solution/acceptance-criteria/<feature>_ac.md
3. Use Given / When / Then
4. Include:
 - Negative cases
 - Boundary conditions
 - Role-based behavior
 - Data state variations

Rules:

- No shallow happy-path-only criteria

3.6 /risk-log-update

File: .claude/commands/risk-log-update.md

You are acting as a Delivery Risk Manager.

Inputs:

- PRD
- Epics
- Stories
- NFRs

Task:

1. Identify delivery, scope, data, and dependency risks
2. Update or create:
04_governance/risks/<feature>_risks.md
3. For each risk include:
 - Description
 - Likelihood
 - Impact
 - Mitigation
 - Owner

Rules:

- Be conservative
- Avoid generic risks

3.7 /decision-record

File: .claude/commands/decision-record.md

You are acting as a Senior Product Leader.

Task:

1. Create an ADR-style decision record
2. Output to:
04_governance/decisions/<decision_name>.md

Structure:

- Context
- Decision
- Alternatives considered
- Rationale
- Consequences
- Open follow-ups

Rules:

- *Decisions must be explicit*
- *Avoid post-hoc justification*

3.8 /release-readiness

File: .claude/commands/release-readiness.md

You are acting as a Release Manager.

Inputs:

- Stories
- Acceptance Criteria
- NFRs
- Risks

Task:

1. Generate a release readiness checklist
2. Output to:
03_delivery/release-notes/<feature>_readiness.md

Include:

- Functional readiness
- Data readiness
- Security/compliance readiness
- Support and ops readiness
- Known limitations

4. How BA / PMs Should actually use this (Weekly Rhythm)

Monday

- /intake-to-prd
- /prd-to-epics

Mid-Sprint

- /epics-to-stories
- /stories-to-ac

Before Sprint Commit

- /nfr-pack

- /risk-log-update

Before Release

- /decision-record
- /release-readiness

This creates **predictable, high-quality product artifacts** with far less cognitive load.

5. Why This Works (Important Insight)

What you are doing here is **encoding product thinking**, not automating writing.

- Claude handles structure and completeness
- Humans handle judgment and trade-offs
- Governance becomes natural, not bureaucratic

This is exactly the shift from *prompting* to *operating* with AI — something most teams never cross.

Closing note: from using AI to operating with it

What changes with Claude Code is not just *how fast* Business Analysts and Product Managers work, but *how consistently* they think and execute.

Most teams begin their GenAI journey with isolated prompts. They ask for a PRD, a summary, a set of user stories. The outputs may look good, but the work remains fragile. Every iteration depends on who is prompting, how much context they remember to provide, and how disciplined they are in reviewing what comes back.

When Claude is used through Claude Code — with skills, commands, and a shared workspace — that fragility disappears.

Product thinking becomes explicit. Standards are no longer tribal knowledge. Decisions, assumptions, risks, and requirements start to live together instead of being scattered across documents, tools, and conversations. The AI stops being a clever assistant and starts behaving like a dependable product operations layer that reinforces how the team works.

For Business Analysts, this means spending less time translating ambiguity into structure and more time validating the *right* structure with stakeholders. For Product Managers, it means moving away from document production as a bottleneck and toward faster, clearer decision-making across discovery, delivery, and release.

The real shift is subtle but profound. You are no longer asking an AI to “help” you. You are designing a system where product work flows predictably, repeatably, and transparently — with AI embedded as part of that system.

Teams that make this shift early will not just move faster. They will think more clearly, align more easily, and scale product execution without scaling chaos. And that, more than any individual feature, is where Claude Code quietly becomes a competitive advantage.

Taking this further

If you found value in how Claude can elevate structured workflows for business analysts and product managers, there is a broader evolution unfolding across everyday productivity tools as well. Claude’s integration into **PowerPoint and Excel** is reshaping how professionals in every function analyze data, build narratives, and communicate insights — all without needing deep technical expertise.

You can explore this next phase of practical AI adoption in my follow-up article: **[Claude in PowerPoint and Excel will change how we work.](#)**

Claude

Anthropic Claude

Claude Skills

Ai Code Assistant

Business Srs



Follow

Written by Ravindra Satyanarayana

36 followers · 29 following

Consultant & strategist helping enterprises adopt GenAI for software engineering, delivery excellence & innovation. Bridging vision with execution.